

Local Market Lab — Technische Dokumentation v0.8.0

*Fortschrittsorientierte Marktanalyse auf deinem eigenen Rechner. Keine Cloud.
Keine Datenweitergabe. Keine Signale. Keine Beratung.*

Inhaltsverzeichnis

1. [Einleitung & Mission](#)
2. [Architektur-Übersicht](#)
3. [Schnellstart](#)
4. [API-Referenz](#)
5. [Portfolio-Engine](#)
6. [Backtest-Engine](#)
7. [Szenarien-Engine](#)
8. [Trading-Game](#)
9. [KI-Prediction](#)

10. [Risk-Analytics](#)
 11. [Bank-Ready & Compliance](#)
 12. [Windows-App](#)
 13. [Ollama-Integration](#)
 14. [Konfiguration](#)
 15. [FAQ](#)
 16. [Troubleshooting](#)
-

1. Einleitung & Mission

Local Market Lab ist eine lokal arbeitende, datenschutzorientierte Workbench für:

- **Portfolioanalyse** — Bewertung, P&L, Allokation
- **Backtesting** — Strategien testen mit Gebühren, Slippage, Benchmarks
- **Szenariosimulation** — Monte-Carlo, Block-Bootstrap, historische Replay
- **Trading-Spiel** — Paper-Trading mit virtuellen Kapital und Rangliste

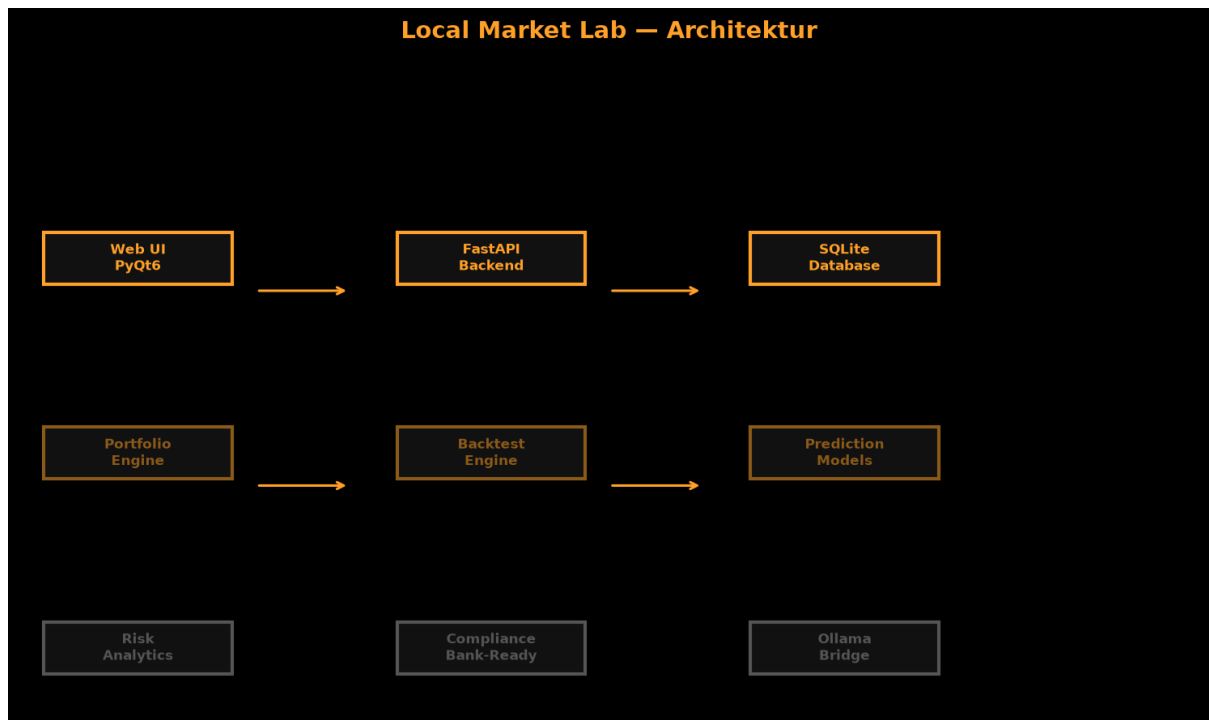
- **KI-Prediction** — 15+ Modelle, lokal, ohne Cloud-Abhängigkeit
- **Risikoanalyse** — VaR, CVaR, Korrelation, Rolling-Metriken

Mission: Institutionelle Methodik privaten Nutzern zur Verfügung stellen, ohne Kompromisse bei Datenschutz und Reproduzierbarkeit.

Design-Prinzipien:

Prinzip	Umsetzung
Lokal	Alles läuft lokal. Keine Cloud erforderlich.
Privacy	Keine Telemetrie, keine Abhängigkeit von externen Services
Reproduzierbarkeit	Jede Berechnung hat Seed, Timestamp, Data-Hash
Transparenz	Alle Methoden dokumentiert, keine Black-Box-Modelle
Decimal-Only	Keine Floats für Geldbeträge, <code>ROUND_HALF_UP</code>
Determinismus	Gleiche Inputs → gleiche Outputs

2. Architektur-Übersicht



2.1 Schichten

Schicht	Technologie	Aufgabe
Web UI	FastAPI + HTML/Canvas	Bloomberg-Terminal-Oberfläche im Browser
Windows App	PyQt6 + pyqtgraph	Native Desktop-Anwendung mit Echtzeit-Charts
API Backend	FastAPI	REST + WebSocket, Port 8322
Datenbank	SQLite	Append-only wo relevant, pro User isoliert
Python-Packages	numpy, requests, sqlite3	Berechnungen ohne externe Abhängigkeiten

2.2 Modulstruktur

```
local-market-lab/
├── apps/
│   ├── api/           # FastAPI REST- und WebSocket-Server
│   ├── cli/           # Typer-Kommandozeilen-Tool
│   ├── web/           # HTML/CSS/Canvas Terminal-UI
│   ├── mcp/           # Model Context Protocol Server
│   └── gui/           # PyQt6 Desktop-App (Windows)
├── packages/
│   ├── core/          # Money, Dates, Hashing
│   ├── domain/        # Entitäten (Instrument, Transaction, ...)
│   ├── storage/       # SQLite-Workspace, State-Singletons
│   ├── ingest/        # CSV-Import, Demo-Daten
│   ├── marketdata/    # Preisreihen, FX, Adapter
│   ├── portfolio/     # Position Engine, Valuation
│   ├── metrics/       # CAGR, Sharpe, Sortino, VaR, CVaR
│   ├── backtest/      # Event-Loop, Strategien
│   ├── scenarios/     # Monte-Carlo, Bootstrap, Prediction
│   ├── artifacts/     # Reproduzierbarkeits-Manifeste
│   ├── compliance/    # Guard, Audit, Bank-Ready
│   ├── reports/       # Markdown-Berichte
│   ├── game/          # Trading-Spiel, Multiplayer-Lobby
│   └── ollama/         # Lokale LLM-Bridge
├── tests/             # 90+ pytest-Tests
└── docs/              # Dokumentation und Grafiken
```

3. Schnellstart

3.1 Installation

```
# Repo klonen
git clone https://github.com/mrmixx-max/local-market-lab.git
cd local-market-lab

# Python 3.10+ erforderlich
python --version

# Dependencies installieren
pip install -e ".[dev]"
```

3.2 Erster Start (Web UI)

```
# API-Server starten
python -m apps.api
|
# Browser öffnen
start http://127.0.0.1:8322/
```

3.3 Erster Start (Windows-App)

```
# Build-Script ausführen
cd windows/src && python build.spec
|
# Oder direkt starten
python -m windows.src.app
```

3.4 Installation Windows-Installer

LocalMarketLab-Setup-v0.8.0.exe aus dem Release herunterladen, ausführen, folgen.

4. API-Referenz

4.1 Gesundheitsprüfung

```
GET /api/v1/health
```

Antwort:

```
{
  "status": "ok",
  "instruments": 4,
  "version": "0.8.0",
  "db_connected": true,
  "uptime_seconds": 42.5
}
```

4.2 Marktdaten

```
GET /api/v1/market/symbols
GET /api/v1/market/prices/{symbol}
```

4.3 Portfolio

```
GET /api/v1/portfolio/{name}?benchmark=IWDA&include_analytics=true
```

4.4 Backtest

```
POST /api/v1/backtest
```

```
{
  "portfolio": {
    "IWDA": 0.6,
    "EIMI": 0.4
  },
  "strategy": "buy_and_hold",
  "fees_bps": 5,
  "slippage_bps": 2
}
```

4.5 Szenario

```
POST /api/v1/scenario
```

```
{  
  "symbols": ["IWDA", "EIMI", "AGGH"],  
  "method": "monte_carlo",  
  "n_simulations": 10000,  
  "horizon_days": 252,  
  "initial_value": 100000,  
  "seed": 42  
}
```

4.6 KI-Prediction

```
POST /api/v1/scenario/forecast/{symbol}
```

```
{  
  "horizon": 30,  
  "method": "ensemble"  
}
```

4.7 Trading-Game

```
POST /api/v1/game/create  
POST /api/v1/game/{game_id}/order  
POST /api/v1/game/{game_id}/tick  
GET /api/v1/game/{game_id}/state  
GET /api/v1/game/leaderboard
```

4.8 Compliance

```
GET /api/v1/compliance/audit-log  
POST /api/v1/compliance/integrity-check  
GET /api/v1/compliance/report
```

5. Portfolio-Engine

5.1 Bewertung

Die Portfolio-Engine nutzt ausschließlich `Decimal`-Werte für Geldbeträge. Float-Eingaben werden beim Parsing abgelehnt.

FX-Policy: Fehlende Wechselkurse erzeugen einen `incomplete`-State — niemals stille 1:1-Konvertierung.

5.2 Corporate Actions

- **Splits:** Automatische Anpassung der Stückzahl
- **Dividenden:** Cash-Dividenden als separate Transaktion
- **Chronologische Reihenfolge:** Alle Aktionen werden in Reihenfolge verarbeitet

5.3 Allokation

```
GET /api/v1/portfolio/{name}?include_analytics=true
```

Liefert `allocation` nach `asset_class` aus der `instruments`-Tabelle.

6. Backtest-Engine

6.1 Verfügbare Strategien

Strategie	Beschreibung
buy_and_hold	Einmal kaufen, nicht verkaufen
periodic_rebalance_63	Alle 63 Tage (Quartal) ausbalancieren
momentum_20	20-Tage-Momentum, Top-Asset kaufen
mean_reversion_20	20-Tage-Mean-Reversion

6.2 Metriken

- **CAGR**: Compounded Annual Growth Rate (252 Tage annualisiert)
- **Max Drawdown**: Maximaler Verlust vom Peak
- **Sharpe Ratio**: Risikoadjustierte Rendite ($R_f=0$)
- **Sortino Ratio**: Wie Sharpe, aber nur Downside-Volatilität
- **Calmar Ratio**: CAGR / Max Drawdown

7. Szenarien-Engine

7.1 Methoden

Methode	Seed	Beschreibung
Monte-Carlo	ja	iid Normal>Returns
Block-Bootstrap	ja	Zufallsblöcke aus historischen Returns
Historical-Replay	ja	Ein historisches Jahr verwenden

7.2 Ausgabe

Alle Methoden liefern: - Perzentile (P05, P25, P50, P75, P95) - Verlustwahrscheinlichkeit -
Disclaimer: Keine Prognose, nur Szenario

8. Trading-Game

8.1 Challenges

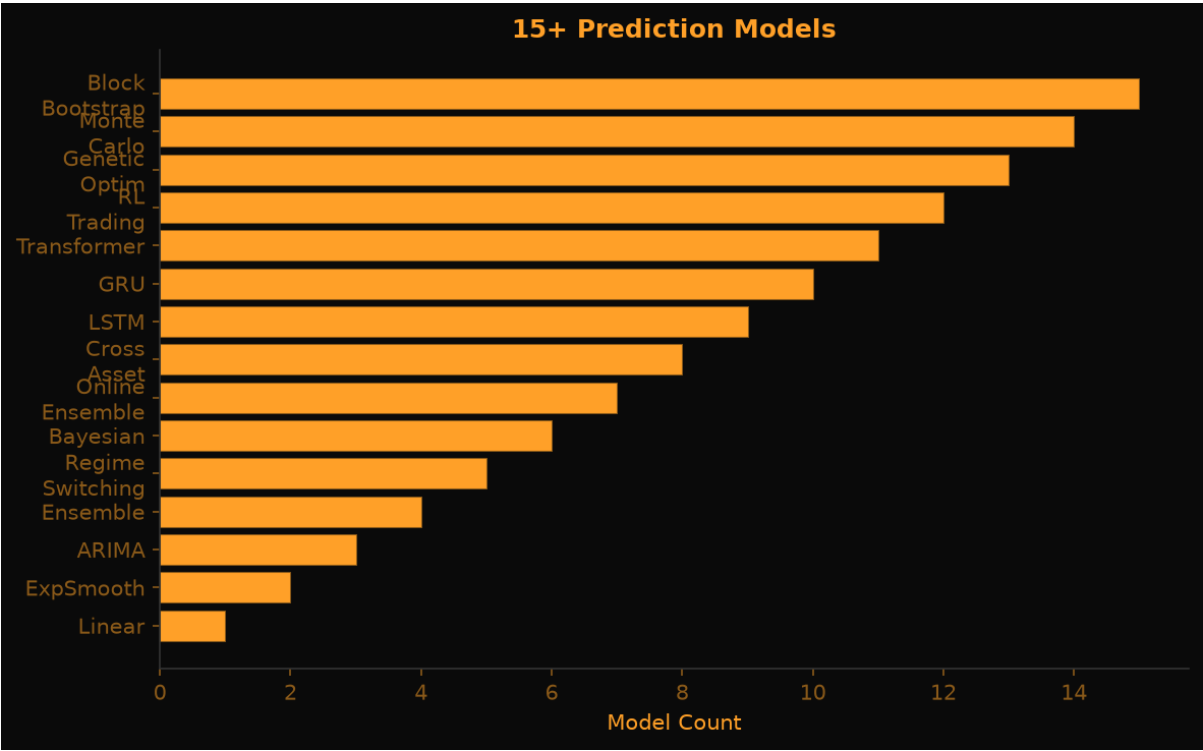
Challenge	Ziel
beat_market	Benchmark (IWDA) schlagen
low_volatility	Max. Volatilität < 10%
income_generator	Monatliche Dividenden
max_sharpe	Max. Sharpe Ratio
min_volatility	Min. Portefeuille-Volatilität
beat_benchmark_by_5pct	5% über Benchmark

8.2 Leaderboard

Endgame-Summary pro Spiel: - Total Return, CAGR, Max Drawdown, Sharpe, Sortino - Anzahl Trades, Win Rate - Equity-Curve für Vergleich

9. KI-Prediction

9.1 15+ Modelle



Kategorie	Modelle
Basismodelle	Linear Trend, Holt's ExpSmooth, ARIMA-like, Ensemble

Fortgeschritten	Regime-Switching, Bayesian Trend/Seasonal, Online Ensemble, Cross-Asset
Deep Learning	LSTM, GRU (BPTT, Adam, Gradient Clipping)
Reinforcement Learning	Q-Learning, DQN, REINFORCE
Genetische Optimierung	Feature-Selection, Differential Evolution, NSGA-II
Szenarien	Monte-Carlo, Block-Bootstrap, Historical-Replay

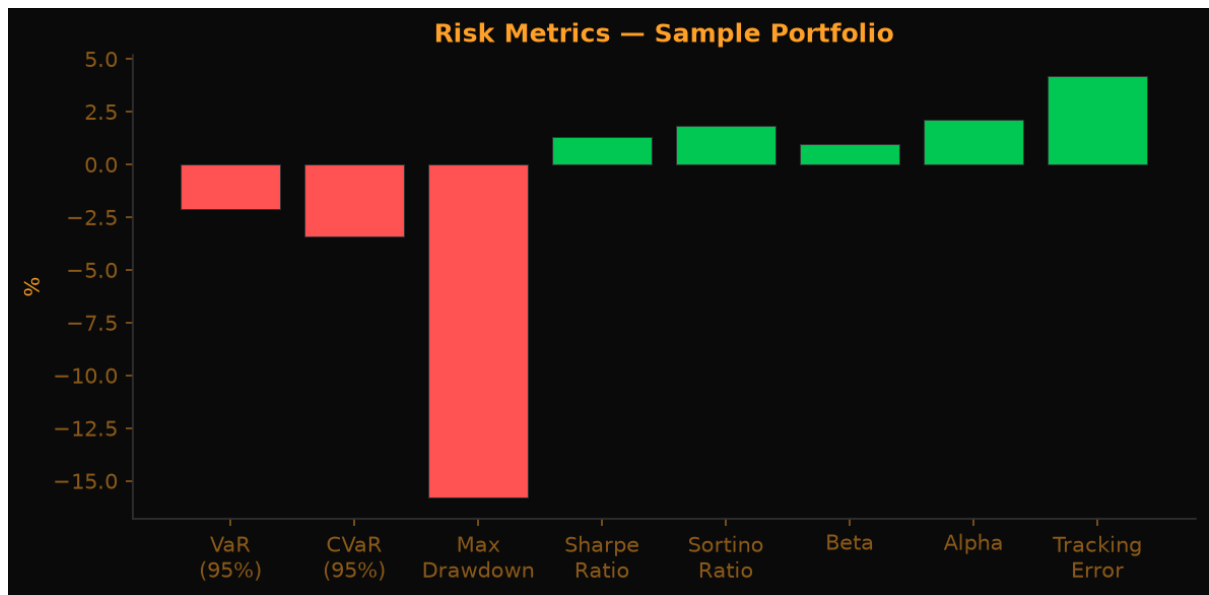
9.2 Ensemble

`ensemble_forecast()` kombiniert drei Basismodelle und gewichtet nach inverser Varianz.

9.3 Konfidenzintervalle

Alle Modelle liefern 68% und 95% Credible/Confidence Intervals.

10. Risk-Analytics



10.1 Metriken

- **VaR (95%)**: Value at Risk, historische Simulation
- **CVaR (95%)**: Expected Shortfall
- **Rolling Sharpe**: 63-Tage rollierend
- **Drawdown Series**: Drawdown als Zeitreihe
- **Performance Attribution**: Pro-Position-Rendite-Beitrag
- **Korrelationsmatrix**: Pearson-Korrelation zwischen Positionen

11. Bank-Ready & Compliance

11.1 Audit-Logger

Append-only Log aller API-Aktionen: - User, Timestamp, Action, Params, Result-Hash - SHA-256-Prüfsummen pro Tabellen-Snapshot

11.2 Data Integrity

Automatische Checksummen für `instruments`, `transactions`, `prices`, `corporate_actions`.

11.3 Compliance-Report (BaFin-Style)

```
{
  "system_version": "0.8.0",
  "audit_log_summary": {...},
  "data_integrity_status": "valid",
  "user_actions_count": 42,
  "risk_flags": []
}
```

11.4 GDPR-Export & -Löschung

- GET `/api/v1/compliance/export/{user}` — JSON-Export
- POST `/api/v1/compliance/delete-account` — Anonymisierung

12. Windows-App

12.1 Installation

1. LocalMarketLab-Setup-v0.8.0.exe herunterladen
2. Installer ausführen
 - Desktop-Verknüpfung optional
3. App starten

12.2 Features

- **6 Tabs**: Markets, Backtest, Scenarios, Game, Ollama, Risk
- **Sidebar**: Watchlist mit Live-Updates
- **Top-Bar**: Branding, Uhr, Verbindungsstatus
- **Statusbar**: Disclaimer
- **Charts**: Candlestick, Line, Histogram, Drawdown (pyqtgraph)
- **Live-Ticks**: WebSocket-Updates im Watchlist

12.3 Architecture

QMainWindow → QTabWidget + QSplitter → Chart-/Dashboard-Widgets

Splash-Screen → Health-Check API → Fallback auf lokale SQLite

13. Ollama-Integration

13.1 Vorbereitung

Ollama installieren und starten:

```
ollama serve  
ollama pull gemma4:latest
```

13.2 API

```
GET /api/v1/ollama/models  
POST /api/v1/ollama/chat  
POST /api/v1/ollama/generate
```

13.3 Prompt-Optimizer

Der Chat-Tab enthält einen eingebauten Prompt-Optimizer mit 5 Tips für bessere Trading-Prompts.

14. Konfiguration

14.1 Umgebungsvariablen

Variable	Default	Beschreibung
LML_HOST	127.0.0.1	API-Server Host
LML_PORT	8322	API-Server Port
LML_DB	./data/marketlab.db	SQLite-Pfad
OLLAMA_HOST	http://localhost:11434	Ollama-Server
LML_CORS_ORIGINS	*	CORS-Origins (kommagetrennt)

14.2 Demo-Daten

```
lml demo
```

Lädt synthetische Preise (IWDA, EIMI, AGGH, BTC) und ein Demo-Portfolio.

15. FAQ

F: Warum keine echten Marktdaten? A: Datenschutz. Echte Daten erfordern API-Keys und senden Abfragen an externe Server. LML funktioniert komplett offline.

F: Kann ich meine eigenen CSV-Dateien importieren? A: Ja, mit `lml import txn` und `lml import prices`.

F: Ist das Finanzberatung? A: Nein. LML ist ausschließlich Forschungs- und Bildungsinvestment. Keine Kauf-/Verkaufsempfehlungen.

F: Warum Python und nicht C++/Rust? A: Lesbarkeit, Reproduzierbarkeit, einfache Installation. Performance ist für die meisten Anwendungsfälle ausreichend.

F: Funktioniert das auf macOS/Linux? A: Ja, die Web-UI und CLI funktionieren plattformunabhängig. Die Windows-App ist Windows-spezifisch.

16. Troubleshooting

Problem	Lösung
429 Too Many Requests	Rate Limiter aktiv — 100 Anfragen/Minute/IP
incomplete bei FX	Fehlender Wechselkurs — manuell importieren oder Währung vermeiden
Circular Import	state.py verwenden, nicht direkt workspace.py
game_id not found	Singleton-Problem — API-Server neu starten
App startet nicht	API-Server prüfen: <code>curl http://127.0.0.1:8322/api/v1/health</code>

Lokale First. Keine Cloud. Keine Kompromisse.

Repository: github.com/mrmixx-max/local-market-lab